

The mounts section ❶ lets us define directories where we can read and write data while the project is running. This section defines a *logs/* directory for the deployed project.

The hooks section ❷ defines actions that are taken at various points during the deployment process. In the build section, we install all the packages that are required to serve the project in the live environment ❸. We also run `collectstatic` ❹, which collects all the static files needed for the project into one place so they can be served efficiently.

Finally, in the deploy section ❺, we specify that migrations should be run each time the project is deployed. In a simple project, this will have no effect when there have been no changes.

The other two configuration files are much shorter; let's write them now.

The `routes.yaml` Configuration File

A *route* is the path a request takes as it's processed by the server. When a request is received by Platform.sh, it needs to know where to send the request.

Make a new folder called *.platform/*, in the same directory as *manage.py*. Make sure you include the dot at the beginning of the name. Inside that folder, make a file called *routes.yaml* and enter the following:

```
.platform/ # Each route describes how an incoming URL will be processed by Platform.sh.
routes.yaml
"https://{default}(":
  type: upstream
  upstream: "ll_project:http"

"https://www.{default}(":
  type: redirect
  to: "https://{default}("

```

This file makes sure requests like *https://project_url.com* and *www.project_url.com* all get routed to the same place.

The `services.yaml` Configuration File

This last configuration file specifies services that our project needs in order to run. Save this file in the *.platform/* directory, alongside *routes.yaml*:

```
.platform/ # Each service listed will be deployed in its own container as part of your
routes.yaml # Platform.sh project.

db:
  type: postgresql:12
  disk: 1024

```

This file defines one service, a Postgres database.